Google Summer of Code 2026 Proposal

Machine Learning for Science (ML4SCI)

High-Energy Physics Simulation (HEPSIM)

Agentic Exclusion Limit Extraction

Submitted to the **Machine Learning for Science (ML4SCI)**

Corresponding Project: **HEPSIM**

Project Size: **350 hours**

Mentors

Steve Mrenna
Fermilab

Konstantin Matchev
University of Alabama

Alex Roman
University of Alabama

Tony Menzo
University of Alabama + Fermilab

Ian Pang
Rutgers University

Personal Details

Name: Srijan Roy

Student: Department of Electronics and Telecommunication Engineering, Jadavpur University

Email: srijanroysr10@gmail.com

Phone: +91 9330961453

GitHub: github.com/SrijanRoy123-github

Resume: Resume Link

Timezone: Indian Standard Time (UTC +5:30)

Contents

Abstract **1**

1. Project Overview **1**

2. Motivation **2**

3. Understanding of the Existing Ecosystem and Project Context **2**

3.1 HEPTAPOD as the main orchestration framework 2

3.2 Orchestral AI as the agent foundation 3

3.3 SModelS as the most relevant reinterpretation database layer 3

3.4 pyhf as a statistical backend where likelihoods are available 4

3.5 Related context from CheckMATE and GAMBIT 4

4. Key Technical Observations and Proposed Implementation Direction **4**

4.1 The problem is best treated as a structured retrieval and matching pipeline 4

4.2 A structured BSM model specification is essential 5

4.3 Retrieval should be split into source-specific adapters 6

4.4 Relevance matching must be explicit and inspectable 6

4.5 Applicability caveats are a first-class deliverable 7

4.6 Audit trails and structured reports should be built into the core workflow 7

4.7 Validation should use known benchmark BSM scenarios 8

5. Planned Code-Level Improvements and Implementation Strategy **9**

6. Proposed Deliverables **10**

7. Work Plan **10**

Phase 1: Environment Setup, Repository Mapping, and Interface Design 11

Phase 2: Retrieval Tools and Relevance Matching 11

Phase 3: Applicability Checks, Audit Trails, and Backend Integration 11

Phase 4: Benchmark Validation, Documentation, and Final Polish 11

8. Timeline **11**

9. Risk Assessment and Mitigation **13**

10. Personal Information **13**

10.1 Communication 14

10.2 Post GSoC 14

11. Contributions and Preparation **15**

12. References and Bibliography **15**

Abstract

This project focuses on automating one of the most repetitive and time-consuming steps in Beyond-the-Standard-Model (BSM) phenomenology: identifying which collider searches already constrain a proposed model, collecting the corresponding exclusion information, and reporting the assumptions under which those limits can or cannot be applied [1]. At present, this task often requires manual literature review across ATLAS and CMS papers, auxiliary material, reinterpretation databases, and statistical resources, followed by case-by-case matching between a model's production and decay structure and the corresponding experimental final states.

The project will build an agentic system inside the HEPTAPOD framework [4, 5]. HEPTAPOD already provides a structured orchestration layer for High Energy Physics workflows through schema-validated tools, auditable execution traces, and human-in-the-loop control [5]. My goal is to extend that framework with limit-oriented tools and workflows that can:

- accept a structured BSM model specification,
- retrieve relevant analyses and database entries from literature and public resources,
- match final states and simplified-model information to the supplied model,
- flag applicability caveats and unsupported assumptions,
- and produce a transparent, structured summary with references and audit logs.

The project will use existing reinterpretation resources rather than attempting to replace them. In particular, it will build on the capabilities of SModelS [7, 9, 10] and, where appropriate, integrate likelihood-aware information through pyhf [11]. The expected result is a working HEPTAPOD extension for automated exclusion-limit compilation, validated on benchmark BSM scenarios such as simplified SUSY and leptoquarks, with open-source code, documentation, and reproducible decision logs [1].

1. Project Overview

The Large Hadron Collider has produced a vast body of exclusion results for new-physics searches, but those limits are distributed across ATLAS and CMS papers, auxiliary material, HEPData-style resources, and reinterpretation databases [1, 13]. For a theorist proposing or studying a new BSM model, determining which searches are relevant and what parameter regions are already excluded remains a largely manual workflow.

The GSoC project *Agentic Exclusion Limit Extraction* is designed to address exactly this problem [1]. The goal is to develop an agentic workflow within HEPTAPOD that, given a model specification containing particle content, production modes, and decay channels, can autonomously:

- search the literature and public limit resources,
- retrieve candidate analyses and structured exclusion information,
- compare those analyses to the final states implied by the user-specified model,
- identify when published assumptions appear compatible or incompatible,
- and produce a structured output with references, caveats, and an auditable reasoning trail.

The project is especially well suited to HEPTAPOD because the framework is already designed for orchestrating HEP workflows through modular tools, validated tool schemas, and explicit execution

boundaries [5]. Rather than writing a monolithic application, I plan to implement a collection of reusable tool modules and agent workflows that can later support broader reinterpretation and phenomenology tasks.

Core project objective: Build a HEPTAPOD-native agentic workflow that automates retrieval, relevance matching, applicability checking, and structured reporting of collider exclusion limits for user-specified BSM models.

2. Motivation

The reinterpretation problem in collider phenomenology has been recognized for years. Experimental analyses are often presented either in the context of specific complete models or in simplified-model form, but applying those results to an arbitrary BSM scenario usually requires additional human work [12, 13]. Simplified-model assumptions may not hold exactly, branching fractions may not be unity, production channels may differ from the experimental benchmark, and the published results may only partially cover the relevant topology.

As discussed in the CheckMATE literature, a fully rigorous reinterpretation may require event generation, detector simulation, and analysis recasting, which is powerful but computationally expensive and difficult to scale across many models or many searches [12]. At the same time, the GAMBIT review highlights that modern BSM inference is increasingly constrained by the practical difficulty of combining many searches, preserving likelihood information, and integrating null results efficiently across large parameter spaces [13].

This project is motivated by the observation that, before any full recasting is attempted, there is already a major bottleneck at an earlier stage:

- finding the relevant searches,
- collecting usable limit information,
- checking whether the analysis assumptions seem applicable,
- and organizing all of that into something transparent and reproducible.

That earlier stage is exactly where an agentic, tool-using workflow can provide practical value. HEPTAPOD already provides the orchestration philosophy needed for such a system: modular tools, structured outputs, explicit validation, and human oversight [5]. My proposal therefore focuses on automating the *search, triage, matching, and reporting* part of reinterpretation, while connecting to established reinterpretation tools such as SModelS and pyhf where appropriate [7, 9, 11].

3. Understanding of the Existing Ecosystem and Project Context

This project sits at the intersection of agent frameworks, HEP workflow orchestration, reinterpretation tools, and statistical backends. A good proposal therefore needs to be grounded not only in the project description, but also in the software ecosystem into which the work will fit.

3.1 HEPTAPOD as the main orchestration framework

The HEPTAPOD framework is the primary technical base for this project [4]. HEPTAPOD is designed to integrate LLMs directly into HEP workflows through schema-validated tools, run-card-driven orchestration, structured execution traces, and human-in-the-loop control [5]. Its current

documented capabilities already include tool interfaces for literature search, particle data lookup, unit conversion, event-generation workflows, and analysis utilities [4].

This is important because the proposed project is not starting from zero. HEPTAPOD already provides:

- a tool-oriented architecture,
- a natural place to add new HEP-specific tools,
- auditable execution traces,
- and an interaction model suitable for transparent scientific workflows.

In other words, my project is naturally framed as a **HEPTAPOD extension** for reinterpretation-oriented automation rather than an entirely new standalone framework.

3.2 Orchestral AI as the agent foundation

HEPTAPOD is implemented on top of *Orchestral AI*, which provides a provider-agnostic and type-safe framework for tool-using agents [6]. Orchestral emphasizes unified tool schemas, deterministic control flow, reproducible execution, sandboxed workspaces, and modular tool integration [6]. These ideas are especially relevant for scientific workflows, where debugging, inspection, and reproducibility matter as much as raw automation capability.

For this project, that means the agentic layer can be designed in a principled way:

- new exclusion-limit tools can be exposed through structured schemas,
- outputs can be normalized before downstream use,
- and every agent step can be logged in a form suitable for human review.

3.3 SModelS as the most relevant reinterpretation database layer

SModelS is an automatic public tool for interpreting simplified-model results from the LHC [7]. It decomposes BSM signatures into simplified-model topologies, matches them to an experimental database, and supports several forms of theory-prediction evaluation and statistical handling [9]. The documentation also shows support for database browsing, missing-topology analysis, likelihood calculations for certain result types, and pyhf-backed signal-region combinations where available [9, 10].

This makes SModelS directly relevant to the proposed work for three reasons:

- it already formalizes a large part of the topology-matching problem,
- it provides a structured database and query model for collider limits,
- and it offers a computationally lighter alternative to full event-level recasting in many scenarios.

For that reason, I expect SModelS to be one of the most useful backends for the project, especially for retrieving and structuring known simplified-model limits and for supporting coverage or missing-topology style reasoning.

3.4 pyhf as a statistical backend where likelihoods are available

pyhf provides a pure-Python implementation of HistFactory-style statistical models and is widely used for likelihood preservation and portable statistical inference [11]. It is not by itself a literature-search or reinterpretation database tool, but it becomes highly relevant when an experimental result exposes a pyhf-compatible workspace or when another tool, such as SModelS, can make use of pyhf-based information [8, 9, 11].

For this proposal, I therefore view pyhf not as the primary engine, but as an **optional statistical execution layer** that can be used when compatible likelihood information is actually available.

3.5 Related context from CheckMATE and GAMBIT

The *CheckMATE* paper and the recent *GAMBIT* review provide broader context for the reinterpretation landscape [12, 13]. CheckMATE illustrates the complexity of full recasting through detector simulation, implemented analyses, and CL_s-style exclusion testing [12]. GAMBIT emphasizes how modern global fits and BSM testing have become increasingly dependent on efficient ways to combine null results, preserve likelihood information, and manage large inference workloads [13].

These references are useful because they clarify the appropriate scope for a 350-hour GSoC project. The most realistic and valuable contribution in one summer is not to replace full recasting frameworks, but to automate the *information retrieval, relevance matching, applicability checking, and transparent reporting* layers that currently consume substantial researcher time.

4. Key Technical Observations and Proposed Implementation Direction

After studying the project description, HEPTAPOD repository and paper, the SModelS documentation, the pyhf documentation, and the surrounding reinterpretation literature, I identified a set of concrete technical observations that shape the implementation plan.

My goal is not to promise a universal solver for all reinterpretation problems. Instead, I want to design a strong first version of an agentic exclusion-limit assistant that is technically realistic, useful immediately, and extensible after GSoC.

4.1 The problem is best treated as a structured retrieval and matching pipeline

The project description naturally decomposes into a multi-stage pipeline:

- represent the user's BSM model in structured form,
- retrieve candidate analyses and database entries,
- normalize their metadata,
- match them to the model's production and decay signatures,
- check applicability assumptions,
- and produce a structured report.

This is a strong fit for HEPTAPOD because each stage can be implemented as a separate tool or tool family, with explicit inputs and outputs rather than ad hoc prompt logic [5].

4.2 A structured BSM model specification is essential

The project description explicitly assumes that the input includes particle content, decay channels, and production modes [1]. That means the first technical requirement is a stable model-specification schema. I plan to define a compact intermediate representation that can capture model identity, benchmark parameters, particle content, production mechanisms, decay structure, and user-facing final-state descriptors. A simplified example is shown below.

Listing 1: A proposed structured BSM model specification used as an intermediate representation for exclusion-limit retrieval and matching.

```
{
  "model_name": "S1_leptoquark",
  "benchmark_point": "BP1",
  "particles": [
    {
      "name": "S1",
      "pdg_id": 9000005,
      "mass_GeV": 1500.0,
      "spin": 0,
      "charge": 0.333,
      "color": "triplet"
    },
    {
      "name": "chi0",
      "pdg_id": 1000022,
      "mass_GeV": 100.0,
      "spin": 0.5,
      "charge": 0.0,
      "color": "singlet"
    }
  ],
  "production_modes": [
    {
      "channel": "pp -> S1 S1~",
      "type": "pair_production",
      "mediator": "QCD"
    }
  ],
  "decay_channels": [
    {
      "parent": "S1",
      "daughters": ["mu+", "jet"],
      "branching_fraction": 0.5
    },
    {
      "parent": "S1",
      "daughters": ["chi0", "jet"],
      "branching_fraction": 0.5
    }
  ],
  "final_state_descriptors": {
    "jets": 2,
    "leptons": 1,
    "photons": 0,
    "met": true,
    "heavy_flavor": false,
    "resonance_structure": ["mu-jet"]
  }
}
```

This representation will serve as the coordination boundary between later tools for retrieval, matching, applicability checks, and reporting. It also improves reproducibility, since the same model specification can be reused across repeated runs and benchmark validation scenarios.

4.3 Retrieval should be split into source-specific adapters

The limit information relevant to this project is heterogeneous by construction. Some useful information lives in literature metadata, some in curated reinterpretation databases, and some in public statistical or auxiliary formats. Rather than treating all sources as one generic search problem, I plan to implement a source-adapter layer with at least the following categories:

- literature and metadata search,
- SModelS-style database queries,
- public result or workspace metadata where available,
- and analysis-summary extraction for report generation.

```
source_adapters = {
    "literature_search": {
        "sources": ["INSPIRE", "arXiv", "HEPData metadata"],
        "inputs": ["model_name", "final_states", "keywords"],
        "outputs": ["paper_id", "title", "analysis_id", "summary", "reference_url"]
    },
    "smodels_database": {
        "sources": ["SModelS database"],
        "inputs": ["topology", "mass_hypothesis", "final_states"],
        "outputs": ["analysis_id", "txname", "upper_limit", "likelihood_info", "coverage_status"]
    },
    "public_workspace_metadata": {
        "sources": ["pyhf workspaces", "public likelihood releases", "auxiliary material"],
        "inputs": ["analysis_id", "dataset_id"],
        "outputs": ["workspace_available", "json_path", "signal_regions", "statistical_model"]
    },
    "analysis_summary_extractor": {
        "sources": ["papers", "auxiliary notes", "database entries"],
        "inputs": ["analysis_id", "document_text"],
        "outputs": ["final_states", "assumptions", "luminosity", "collision_energy", "caveats"]
    }
}
```

Figure 1: Proposed source-adapter structure for heterogeneous exclusion-limit retrieval. The workflow separates literature search, SModelS-style database access, public workspace metadata, and analysis-summary extraction into distinct adapter layers with normalized outputs.

This is important because each source has different structure, reliability, and granularity. An explicit adapter layer will make it easier to normalize outputs and also easier to debug when a particular source is incomplete or ambiguous.

4.4 Relevance matching must be explicit and inspectable

One of the central scientific steps in the project is deciding whether an experimental result is relevant to the supplied model. This should not be left as an opaque language-model judgment. Instead, I want the workflow to produce explicit matching evidence, such as:

- overlap in final-state signatures,

- consistency of production topology,
- consistency or mismatch in decay assumptions,
- and the presence or absence of key requirements such as promptness, invisible states, resonance structure, or simplified branching assumptions.

The output of this stage should therefore be more than a ranked list. It should be a **justified ranking** with structured reasons for acceptance, rejection, or uncertainty.

4.5 Applicability caveats are a first-class deliverable

The project description explicitly asks for validation checks that flag when published assumptions may not apply [1]. I consider this one of the most important design points in the proposal.

A useful system must be able to say not only:

- "this analysis looks relevant,"

but also:

- "this limit assumes a 100% branching ratio,"
- "this result is only valid for a simplified topology not fully matched by the model,"
- "this analysis uses a signal-region or likelihood structure unavailable in the public resources,"
- or "the model appears to fall outside the grid or coverage region."

This caveat-oriented design keeps the project scientifically honest and much more useful than a system that produces only a flat yes/no answer.

4.6 Audit trails and structured reports should be built into the core workflow

The project description requires transparent audit trails recording queries, database hits, and reasoning steps [1]. This is one of the clearest places where HEPTAPOD's orchestration philosophy becomes an advantage [5]. Rather than treating logging as an afterthought, I want every major step to record:

- which tools were called,
- which sources were queried,
- what candidate results were found,
- why specific analyses were retained or discarded,
- and what caveats were attached at the end.

This will make the final output much more trustworthy and also make debugging and validation substantially easier.

4.7 Validation should use known benchmark BSM scenarios

The project description already suggests testing on benchmark scenarios where the relevant limits are known, such as simplified SUSY and leptoquarks [1]. I think this is exactly the right validation strategy.

I therefore plan to use at least two benchmark scenarios:

- one simplified SUSY scenario with well-known topology-driven constraints,
- and one leptoquark-style scenario with a cleaner signature and a different analysis pattern.

A simplified sketch of the proposed benchmark-validation setup is shown below.

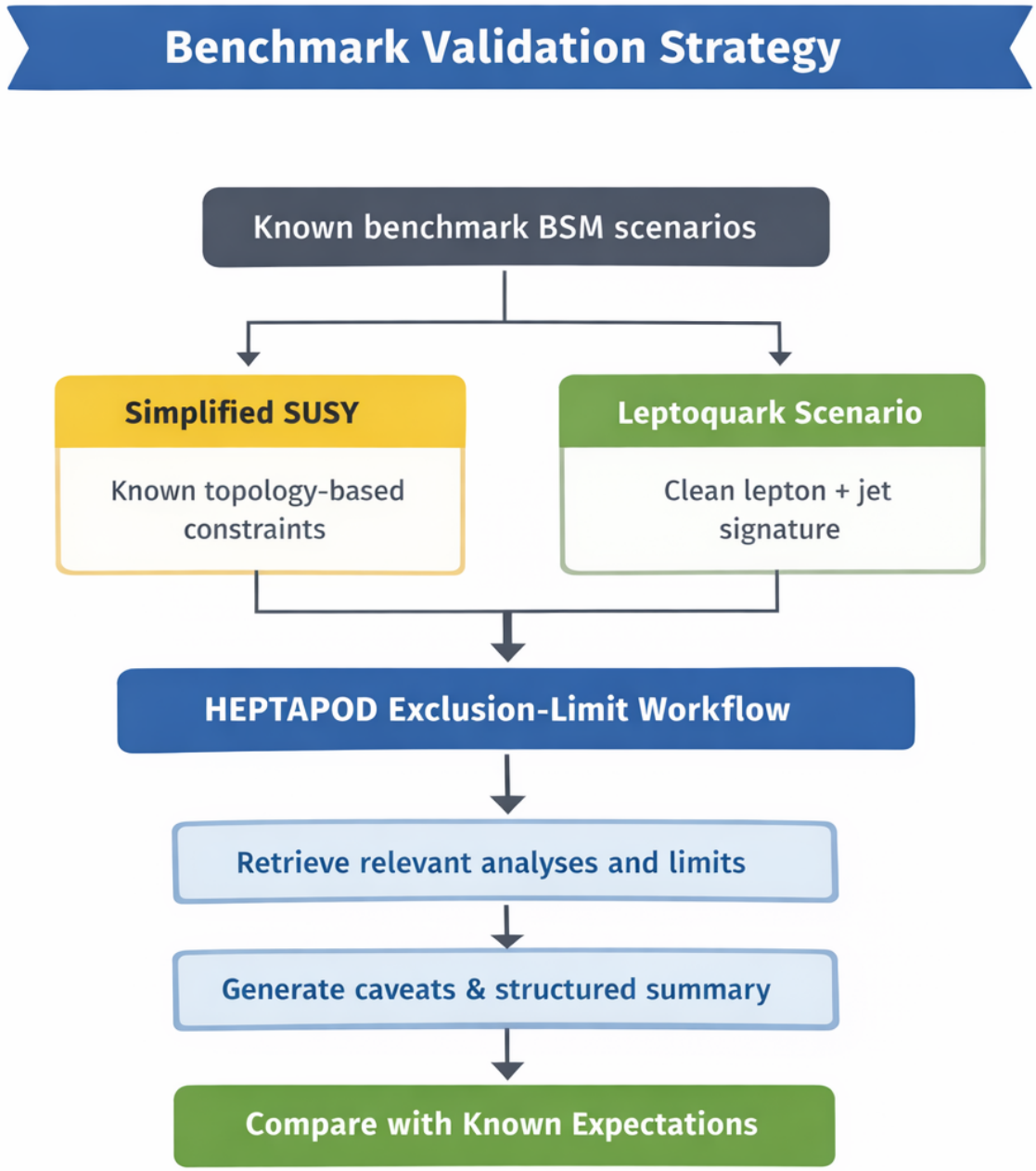

Figure 2: Proposed benchmark-validation setup for the exclusion-limit workflow. The validation stage uses at least two known BSM scenarios, such as simplified SUSY and leptoquarks, to test whether the system retrieves the main relevant searches, reports useful caveats, and produces structured summaries aligned with known expectations.

The goal of validation will not be absolute reinterpretation perfection. Instead, it will be to show that the system can reliably retrieve the main relevant searches, identify useful database-backed limits, and produce caveat-aware summaries that align with known expectations for representative benchmark scenarios.

The goal of validation will not be absolute reinterpretation perfection. Instead, it will be to show that the system can reliably retrieve the main relevant searches, identify useful database-backed limits, and produce caveat-aware summaries that align with known expectations.

5. Planned Code-Level Improvements and Implementation Strategy

The technical observations above motivate a concrete implementation plan inside the HEPTAPOD ecosystem. My approach is to implement the project as a set of modular tools and workflows rather than a single monolithic script. This will make the work easier to test, easier to review, and more reusable after GSoC.

The main code-level improvements and additions I plan to make are:

1. **Introduce a structured BSM model-specification schema.** I plan to define a stable intermediate representation for new particles, masses, production modes, decay chains, branching assumptions, and signature-level descriptors. This schema will act as the formal input to the exclusion-limit workflow.

2. **Add source-specific HEPTAPOD tools for exclusion-limit retrieval.** These tools will query literature and public limit resources, normalize the outputs into a consistent structure, and return machine-readable records rather than free-form text.

3. **Implement a relevance-matching layer between model specifications and candidate analyses.** The system should explicitly compare model-level signatures against analysis-level metadata such as final states, topologies, and experimental assumptions. My goal is to make this comparison inspectable rather than implicit.

4. **Add applicability-check modules and caveat generation.** This part of the workflow will flag assumption mismatches such as incompatible branching-ratio assumptions, unsupported topologies, or results that appear to be outside the covered parameter grid.

5. **Integrate SModelS-oriented database access where useful.** Since SModelS already formalizes topology decomposition and database querying, I plan to use it wherever it can provide structured, validated information more reliably than ad hoc extraction.

6. **Support pyhf-aware statistical metadata where available.** For analyses or resources that expose pyhf-compatible workspaces or related metadata, I plan to connect the workflow so that the final report can clearly state when a stronger statistical interpretation is possible.

7. **Implement explicit audit logging and decision tracing.** Every important search, ranking, filtering, and applicability decision should be recorded in a structured log so that the final report remains reproducible and easy to inspect.

8. **Provide benchmark workflows and examples for at least two known BSM scenarios.** I want the final system to include reusable examples that demonstrate the full pipeline from model specification to structured limit summary on benchmark cases.

9. **Add tests, documentation, and example outputs.** Because the project is intended to become part of an open-source HEP toolkit, I want the deliverables to include not only code but also examples, usage documentation, and tests for the most important components.

Overall, success here means that the project delivers a reliable first version of an exclusion-limit assistant that is modular, auditable, and useful immediately, while also being clearly extensible for more ambitious reinterpretation workflows later.

6. Proposed Deliverables

Core Deliverables

- **Structured BSM model-specification format.** A reusable input schema for particle content, masses, production modes, decay channels, and signature-level descriptors, suitable for agentic workflows inside HEPTAPOD.
- **HEPTAPOD tool modules for exclusion-limit retrieval.** New tool components for querying literature and public exclusion resources and returning normalized machine-readable outputs [1, 4].
- **Analysis relevance-matching workflow.** A transparent matching layer that compares candidate analyses and database entries against the user-specified BSM model and ranks them with structured reasons.
- **Applicability-check and caveat system.** A validation layer that flags assumption mismatches, unsupported topologies, grid-coverage issues, or other reasons a published result may not transfer cleanly to the supplied model [1].
- **Audit trail and decision log.** Structured recording of search queries, database hits, filtering decisions, and final reasoning summaries, aligned with HEPTAPOD's auditable workflow philosophy [5].
- **SModelS-oriented backend integration.** Practical integration of SModelS-based database or topology information wherever it strengthens limit retrieval and structured reporting [7, 9].
- **Validation on at least two benchmark scenarios.** End-to-end evaluation on well-characterized BSM examples such as simplified SUSY and leptoquarks, with explicit comparison to known relevant limits [1].
- **Open-source documentation and examples.** Documentation describing setup, workflow usage, benchmark examples, output interpretation, and limitations.

Stretch Goals (if time permits)

- Limited pyhf-aware statistical support for compatible analyses or workspaces.
- Coverage-gap reporting that identifies regions where no clearly applicable search is found.
- Additional source adapters or richer structured export formats for downstream phenomenology workflows.

7. Work Plan

The project is organized into four major stages. The aim is to move from a stable infrastructure baseline to validated end-to-end workflows on benchmark BSM scenarios.

Phase 1: Environment Setup, Repository Mapping, and Interface Design

- Set up the HEPTAPOD development environment.
- Study the current tool architecture and identify the cleanest integration points.
- Review the SModelS and pyhf interfaces most relevant to the project.
- Design the structured BSM model-specification format and the initial normalized output schema for retrieved analyses.

Outcome: Working development environment, architecture map, and clearly defined input/output schemas.

Phase 2: Retrieval Tools and Relevance Matching

- Implement initial exclusion-limit retrieval tools and source adapters.
- Normalize retrieved metadata into a consistent intermediate format.
- Implement the first relevance-matching and ranking workflow between model specifications and candidate analyses.
- Begin generating structured candidate-analysis summaries.

Outcome: First working pipeline from model specification to ranked candidate analyses.

Phase 3: Applicability Checks, Audit Trails, and Backend Integration

- Add caveat-generation logic and applicability checks.
- Record structured decision logs and audit traces for every major workflow stage.
- Integrate SModelS-oriented database support for stronger topology-aware matching.
- Add pyhf-aware statistical metadata where appropriate and available.

Outcome: Caveat-aware, auditable workflow with stronger structured backends.

Phase 4: Benchmark Validation, Documentation, and Final Polish

- Run the workflow on at least two benchmark BSM scenarios.
- Compare the generated summaries against known relevant searches and limits.
- Refine ranking quality, reporting clarity, and tool robustness.
- Write documentation, examples, and final technical summaries.

Outcome: Validated HEPTAPOD extension with documentation, examples, and benchmark outputs.

8. Timeline

Period	Planned Work	Checkpoint / Deliverable
Community Bonding Period	Set up the local development environment, study the HEPTAPOD architecture and the relevant documentation, read the SModelS and pyhf interfaces in detail, and finalize the detailed implementation scope with mentors.	**Working setup** and architecture map
Week 1	Define the structured BSM model-specification format and the normalized analysis-record schema that later tools will use.	**Input/output schema draft**
Week 2	Design the first source-adapter interfaces for literature and public exclusion resources; decide on the core internal data flow.	**Source adapter plan**
Week 3	Implement the first retrieval tools inside HEPTAPOD and test normalized outputs on a small set of known analyses.	**Initial retrieval tools**
Week 4	Implement relevance matching between model signatures and candidate analyses; begin ranking and structured summary generation.	**First end-to-end prototype**
Week 5	Improve ranking quality, add structured evidence for why an analysis is considered relevant or uncertain, and start small-scale validation.	**Ranked candidate summaries**
Week 6	Add applicability checks and caveat-generation rules for branching assumptions, unsupported topologies, or coverage mismatches.	**Midterm deliverable: caveat-aware workflow**
Week 7	Implement explicit audit logging for queries, database hits, filtering decisions, and final report generation.	**Audit trail support**
Week 8	Integrate SModelS-based database or topology-aware support where it improves structured limit retrieval and reporting.	**SModelS-oriented backend integration**
Week 9	Add optional pyhf-aware metadata support for compatible resources and improve report structure for statistical annotations.	**Statistical metadata improvements**
Week 10	Validate the workflow on a simplified SUSY benchmark scenario and compare the retrieved results against known relevant limits.	**Benchmark validation 1**
Week 11	Validate the workflow on a second benchmark scenario such as a leptoquark model and refine matching, caveats, and coverage reporting.	**Benchmark validation 2**
Week 12	Clean up code, add documentation and usage examples, write final summaries, and prepare the final mergeable contribution.	**Documentation and final code polish**
Final Phase	Incorporate mentor feedback, finalize reporting examples, improve documentation clarity, and summarize conclusions and future directions.	**Final report** and cleaned documentation

9. Risk Assessment and Mitigation

Like most research-oriented tooling projects, this work has a few important practical risks. I summarize them below together with the mitigation strategy I would follow.

- **Public sources are heterogeneous and incomplete.** Literature pages, reinterpretation databases, and statistical resources do not all expose the same structure or the same level of detail. To mitigate this, I will use a source-adapter design with normalized outputs, explicit missing-field handling, and careful prioritization of the most structured sources first.
- **Applicability of published limits is inherently model-dependent.** A search may look relevant at the final-state level while still relying on assumptions that do not transfer cleanly to the supplied model. To mitigate this, the workflow will treat caveat generation as a first-class deliverable and will prefer transparent partial answers over overconfident automation.
- **LLM-based reasoning can introduce ambiguity or overreach.** Since the project includes agentic reasoning, there is a risk of unsupported matching decisions. To mitigate this, I will keep the workflow grounded in structured metadata, explicit tool outputs, and auditable reasoning logs rather than relying on free-form judgments alone.
- **Scope may grow too broad for a 350-hour project.** The reinterpretation landscape is large, and it would be easy to promise universal coverage. To mitigate this, I will prioritize the core path first: structured input schema, source adapters, relevance matching, caveat generation, audit trails, and validation on two benchmark scenarios. Broader source coverage and more advanced statistical integration will remain stretch goals.
- **Validation may show only partial agreement with known expectations.** That is a real possibility because the project is not intended to replace full detector-level recasting. To mitigate this, the evaluation criteria will focus on whether the workflow retrieves the main relevant searches, reports useful caveats, and produces transparent structured summaries, not whether it perfectly reproduces every downstream exclusion conclusion.
- **Some backends may be harder to integrate than expected.** For example, richer pyhf-aware support may depend strongly on the availability and quality of compatible public resources. To mitigate this, I will keep pyhf support optional and backend-aware rather than making it a hard dependency for the core workflow.

Overall, the main mitigation strategy is to keep the project modular, baseline-driven, transparent, and validation-focused. That ensures the project remains useful even if some more ambitious extensions are only partially completed.

10. Personal Information

I am Srijan Roy, an undergraduate student in the Department of Electronics and Telecommunication Engineering at Jadavpur University, Kolkata. Over the last few years, I have developed a strong interest in research-oriented software development, especially at the intersection of machine learning, scientific computing, and reproducible technical systems. What attracts me most to open source is the opportunity to contribute to real research problems, understand nontrivial codebases, and build tools that remain useful beyond a single course or short-term project.

My background is shaped by two main tracks. The first is research in machine learning and computer vision, where I have worked on spectral image analysis, segmentation, remote sensing, deepfake detection, and video understanding. That work has required building training and evaluation pipelines, benchmarking models, analyzing performance across datasets, and writing reproducible

research code. I have also co-authored research work on spectral representation learning, and I maintain a Google Scholar profile listing my research output [15, 16].

The second track is broader software and open-source development. I am currently a part-time contributor to the Formal Conjectures Project at Google DeepMind, where I work with Lean and formalized mathematics. I have also served as the Web Development Lead of the IoT Applications Club at Jadavpur University, where I contributed to the design and development of the club website with the development team [17].

From the technical side, I am comfortable with Python, C++, C, JavaScript, HTML, CSS, Lean, and PHP. I regularly work with PyTorch, NumPy, scikit-learn, OpenCV, Matplotlib, Linux-based development environments, and version-controlled collaborative workflows. More broadly, I enjoy projects that require both implementation and careful reasoning. My research background has trained me to think in terms of experiments, baselines, comparisons, and measurable outcomes, while my software experience has made me comfortable working inside unfamiliar codebases and improving them incrementally. I believe this combination is a strong fit for a project like this one, which requires both engineering discipline and scientific caution.

10.1 Communication

I value regular communication and iterative feedback, especially for research-oriented software projects where design decisions and validation criteria matter as much as code itself. For a project like this one, I believe it is important to keep mentors informed not only about completed work, but also about intermediate results, ambiguities, and trade-offs that may benefit from early discussion.

I am based in India and work in Indian Standard Time (UTC +5:30). During the GSoC period I will be able to dedicate full-time effort to the project. I am comfortable with communication over email, issue trackers, GitHub discussions, and synchronous meetings when needed. I also adapt easily to the communication preferences of the mentor team and can adjust my schedule when cross-time-zone coordination is useful.

My general communication style is to keep progress visible and technically concrete. I prefer sharing intermediate results, small design notes, and reproducible benchmarks early rather than waiting until every detail is polished. Since this project involves ranking, caveat generation, and validation on benchmark scenarios, I believe that surfacing partial findings early will make the final result much stronger.

10.2 Post GSoC

I would like this project to be the beginning of a longer engagement rather than a one-time summer contribution. The work proposed here fits naturally into a broader future direction for HEPTAPOD and ML4SCI: agentic tools for scientific reasoning, literature-grounded model interpretation, and reusable HEP automation workflows.

Even if the GSoC period focuses on a well-scoped first implementation, I expect the results to naturally lead to follow-up work. Examples include better source adapters, richer pyhf-aware support, broader coverage-gap analysis, and integration with other phenomenology or reinterpretation tasks. I would be very interested in continuing to contribute after GSoC, especially if the initial system proves useful to the mentors and the broader community.

More broadly, I would like to remain involved with the ML4SCI and HEPSIM ecosystem, keep learning from the maintainers and mentors, and continue contributing where my skills are useful. I see this proposal not only as a summer project, but as a strong opportunity to contribute to a scientific software effort whose long-term value extends well beyond the immediate deliverables.

11. Contributions and Preparation

I have already begun preparing for this project by studying the official project description, the HEPTAPOD repository and paper, the Orchestral AI paper, the SModelS website and documentation, the pyhf documentation, and related reinterpretation literature such as CheckMATE and the GAMBIT review [1, 4, 5, 6, 7, 9, 11, 12, 13]. This preparation has helped me refine the project scope into something that is both technically realistic and scientifically useful for a 350-hour GSoC project.

I have also approached the project with an implementation mindset rather than only a proposal-writing mindset. In particular, I have focused on understanding:

- how HEPTAPOD exposes domain-specific tools to an agent,
- why SModelS is the most natural reinterpretation-oriented backend for the first version of this work,
- where pyhf fits into the workflow and where it does not,
- and why transparent applicability caveats and audit trails are essential rather than optional.

This preparation gives me a clear technical direction going into the project: build reusable, structured, auditable components first, validate them on known benchmark scenarios, and keep the workflow honest about what it can and cannot conclude automatically.

I have also completed the official test task for this project and organized my work in a dedicated repository. For completeness, I include both the official test link and the repository link below:

- **Official Project Test:** Project Test Link
- **Test Repository:** GitHub Repository for Test Task

12. References and Bibliography

References

[1] Machine Learning for Science, *Agentic Exclusion Limit Extraction*, GSoC 2026 project page, online. Available: `https://ml4sci.org/gsoc/2026/proposal_HEPSIM4.html`

[2] Machine Learning for Science, *ML4SCI Official Website*, online. Available: `https://ml4sci.org/`

[3] Machine Learning for Science, *GSoC 2026 Projects Related to HEPSIM*, online. Available: `https://ml4sci.org/gsoc/projects/2026/project_HEPSIM.html`

[4] Tony Menzo et al., *HEPTAPOD Repository*, GitHub repository, online. Available: `https://github.com/tonymenzo/heptapod`

[5] T. Menzo, A. Roman, S. Gleyzer, K. Matchev, G. T. Fleming, S. Höche, S. Mrenna, and P. Shyamsundar, *HEPTAPOD: Orchestrating High Energy Physics Workflows Towards Autonomous Agency*, arXiv:2512.15867, 2025. Available: `https://arxiv.org/abs/2512.15867`

[6] A. Roman and J. Roman, *Orchestral AI: A Framework for Agent Orchestration*, arXiv:2601.02577, 2026. Available: `https://arxiv.org/abs/2601.02577`

[7] SModelS Collaboration, *SModelS: A tool for interpreting simplified-model results from the LHC*, official website, online. Available: `https://smodels.github.io/`

[8] SModelS Collaboration, *SModelS Documentation: Installation and Deployment*, online documentation. Available: https://smodels.readthedocs.io/en/latest/Installation.html

[9] SModelS Collaboration, *SModelS Documentation: Using SModelS*, online documentation. Available: https://smodels.readthedocs.io/en/latest/UsingSModelS.html

[10] SModelS Collaboration, *SModelS Documentation: SModelS Tools*, online documentation. Available: https://smodels.readthedocs.io/en/latest/SModelSTools.html

[11] pyhf developers, *pyhf Documentation v0.7.6*, online documentation. Available: https://pyhf.readthedocs.io/en/v0.7.6/

[12] M. Drees, H. Dreiner, D. Schmeier, J. Tattersall, and J. S. Kim, *CheckMATE: Confronting your Favourite New Physics Model with LHC Data*, arXiv:1312.2591, 2014. Available: https://arxiv.org/abs/1312.2591

[13] P. Athron et al. (GAMBIT Collaboration), *Global fits and the search for new physics: past, present and future*, arXiv:2512.14109, 2025. Available: https://arxiv.org/abs/2512.14109

[14] LHC Reinterpretation Forum, *Reinterpretation of LHC Results for New Physics: Status and Recommendations after Run 2*, arXiv:2003.07868, 2020. Available: https://arxiv.org/abs/2003.07868

[15] B. Das, A. Ali, S. Roy, S. Anam, K. Djemal, and S. S. Chaudhuri, *Spectral Representation Learning in Color Space for Deforestation Mapping with Limited Supervision*, in *2025 IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*, pp. 1192–1198, 2025. Available: https://doi.ieeecomputersociety.org/10.1109/ICTAI66417.2025.00174

[16] Srijan Roy, *Google Scholar Profile*, online. Available: https://scholar.google.com/citations?hl=en&user=SFW2ElkAAAAJ

[17] IoT Applications Club, Jadavpur University, *IOTA Website*, online. Available: https://www.iotaju.com/HOME/homepage-of-iota-main/home.html